LAB REPORT

Wireshark: Traffic Analysis

Network Traffic Investigation and Anomaly Detection

Blue Team, SOC Level 1, Zen Mier

1. Objectives

This lab was completed as part of a structured blue team training path on TryHackMe. The goal was to develop practical skills in reading and interpreting network traffic captures, understanding not just what data is moving across a network, but whether any of that activity looks suspicious or harmful.

By the end of the lab, the following objectives were expected to be met:

- Identify different types of network scanning activity performed by an attacker before launching an attack.
- Detect ARP poisoning, a technique used to intercept network communication between two machines.
- Determine the identity of devices on a network using protocol traffic such as DHCP, NetBIOS, and Kerberos.
- Recognize covert communication channels hidden inside normal-looking DNS and ICMP traffic.
- Analyze unencrypted network protocols (FTP and HTTP) to identify exposed usernames, passwords, and file transfers.
- Decrypt and inspect encrypted HTTPS traffic using a provided key file to uncover hidden activity.
- Locate credentials transmitted in plain text across the network.
- Generate firewall blocking rules directly from suspicious traffic captured in Wireshark.

2. What Was Done

The lab provided pre-captured network traffic files (known as PCAP files) essentially recordings of all data packets that passed through a network over a period of time. The task was to open these recordings in Wireshark, apply filters to focus on specific types of traffic, and draw conclusions about what was happening on the network.

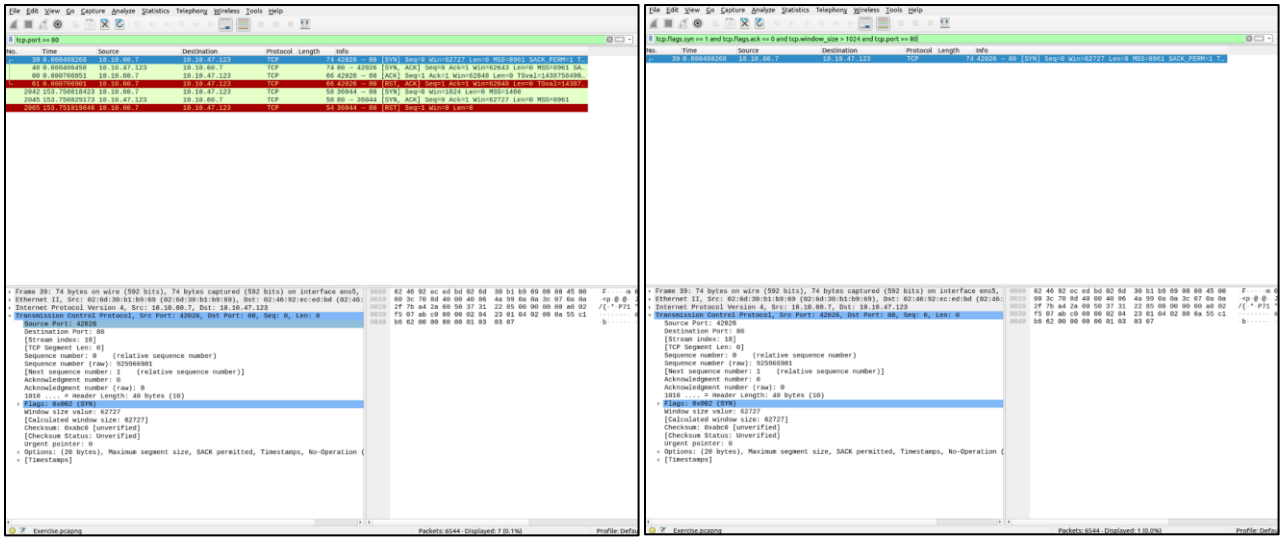
The investigation was broken into several areas, each focused on a different type of suspicious activity. All findings were based entirely on evidence found within the traffic files.no assumptions were made without supporting packet data.

Detecting Network Scanning

The first area of investigation focused on identifying scanning activity, the process an attacker uses to map out a network before deciding where to attack. Three different scanning methods were identified in the traffic:

- TCP Connect Scan, the attacker fully completes a connection to each port to check if it is open. This leaves a clear footprint in the traffic because every connection is completed normally.
- SYN Scan, a faster, quieter method where the attacker only sends the first part of a connection request without completing it. This is harder to detect on a live system but is still visible in a traffic capture.
- UDP Scan, a check for open UDP ports, which are used by services like DNS. These show up differently in traffic because UDP does not use a formal connection process.

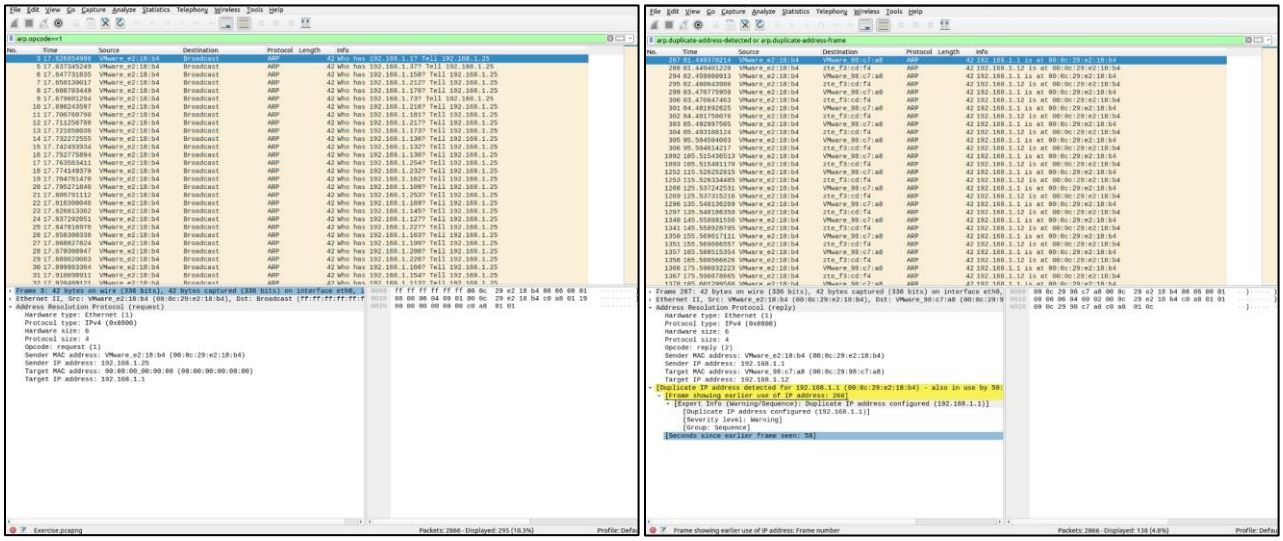
Wireshark filters were applied to isolate each type of scan, allowing the source IP address and the ports being proved to be clearly identified.



ARP Poisoning and Traffic Interception

ARP is a protocol that maps an IP address to the physical address (MAC address) of a device on the same network. An attacker can abuse this by sending out fake ARP messages to trick other devices into sending their traffic through the attacker's machine, a technique called ARP poisoning or a Man-in-the-Middle attack.

In this section, duplicate ARP responses were identified in the traffic, multiple devices claiming to own the same IP address. This is the key indicator of ARP poisoning. The attacker's MAC address was identified, along with the two victim devices whose communication was being intercepted.

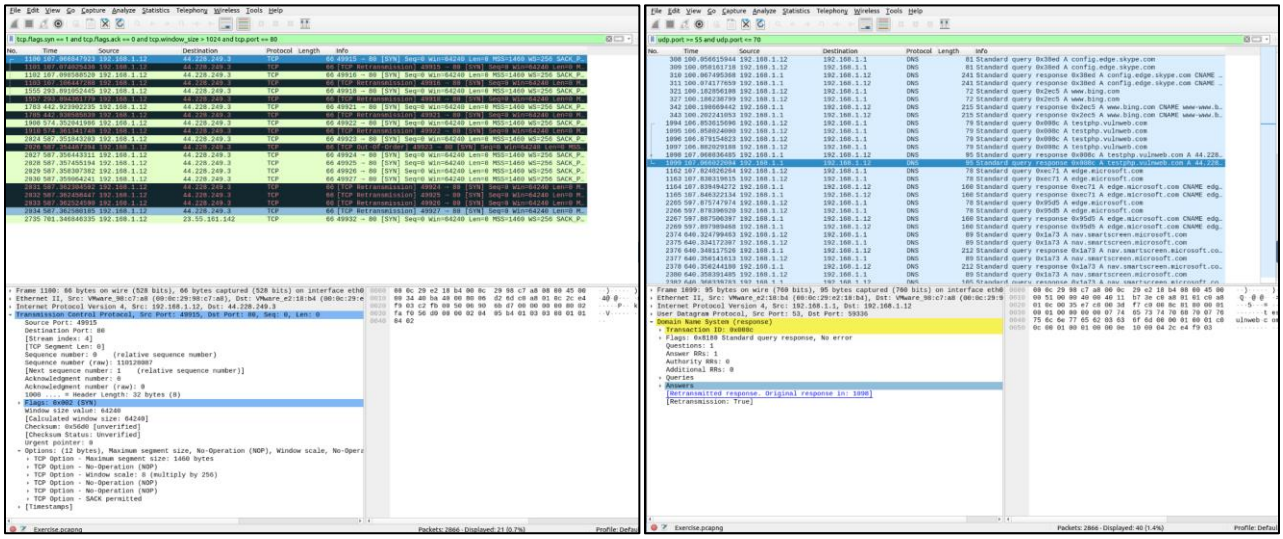


Identifying Devices on the Network

Even without direct access to a network, it is possible to identify what machines are present by observing how they behave in traffic. Three protocols were used for this:

- DHCP traffic, when a device joins a network, it broadcasts a request for an IP address. This request often includes the device's hostname, operating system, and vendor information.
- NetBIOS traffic, an older Windows protocol that broadcasts device names across the local network. These broadcasts reveal hostnames and workgroup names.
- Kerberos traffic, an authentication protocol used in Windows environments. Kerberos packets contain account names and domain information, making it possible to identify specific users who were logged in during the capture period.

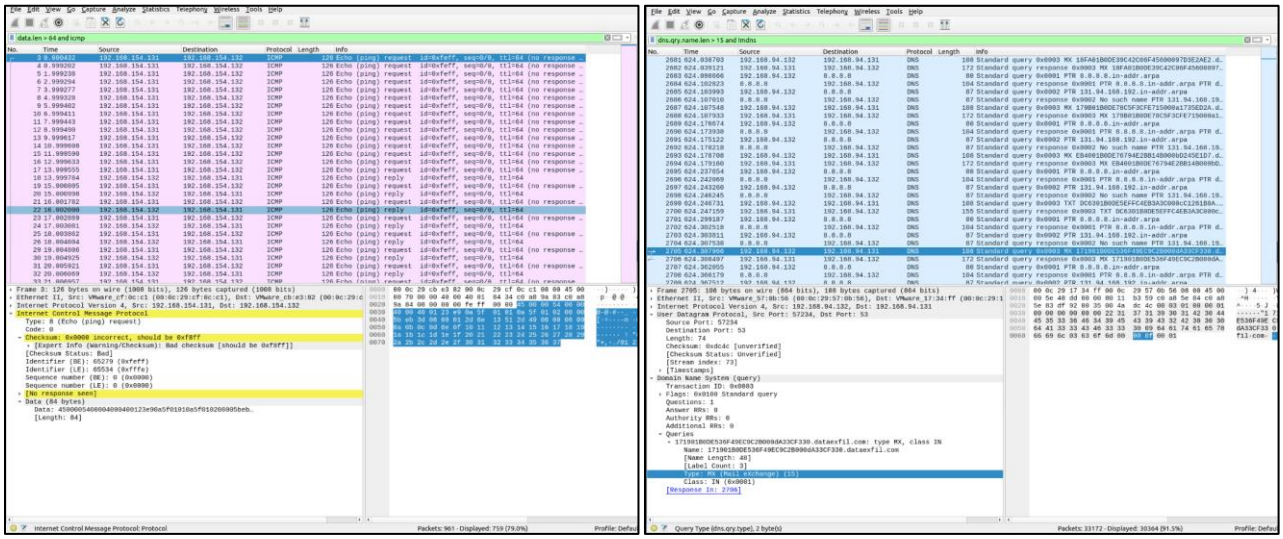
By filtering each protocol and reading the packet contents, a list of devices, hostnames, and usernames active during the capture window was built.



Detecting Tunneling Traffic

Tunneling is a method where an attacker hides data inside a protocol that is normally allowed through a firewall, such as DNS (used for translating website names to IP addresses) or ICMP (used for simple network tests like ping). By encoding secret data inside these protocols, an attacker can move information in or out of a network without raising immediate suspicion.

In the traffic capture, abnormally large DNS queries and unusually frequent ICMP packets were identified. Normal DNS queries are short, a domain name and a response. The captured DNS packets contained long, encoded strings that had no relation to legitimate domain lookups, which is a strong indicator of data being smuggled through DNS. Similarly, the ICMP packets carried data payloads that were far larger than what a standard ping would produce.



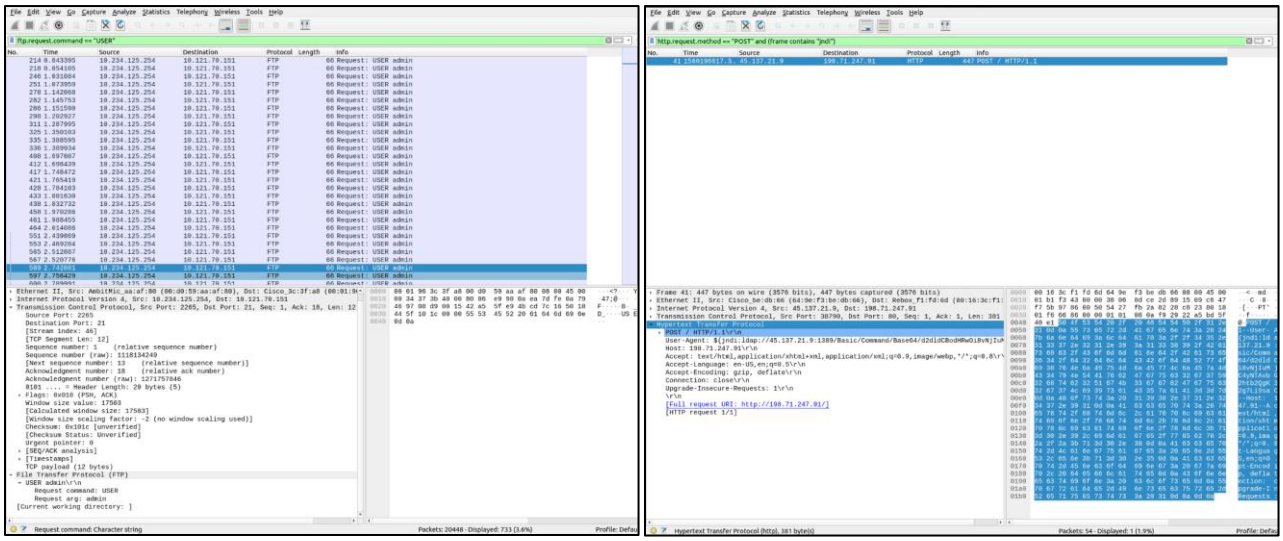
Analyzing Unencrypted Protocol Traffic (FTP and HTTP)

FTP (File Transfer Protocol) and HTTP (the protocol behind unencrypted websites) transmit data in plain text, meaning anyone who can capture the traffic can read it directly, including usernames and passwords.

In the FTP section, the full login sequence was visible in the packets, the username and password were readable without any decoding. File transfer commands and the names of files being moved were also captured.

In the HTTP section, form submissions and login requests were examined. Because the connection was not encrypted, the data entered into web forms including credentials, was visible in clear text within the packet payload.

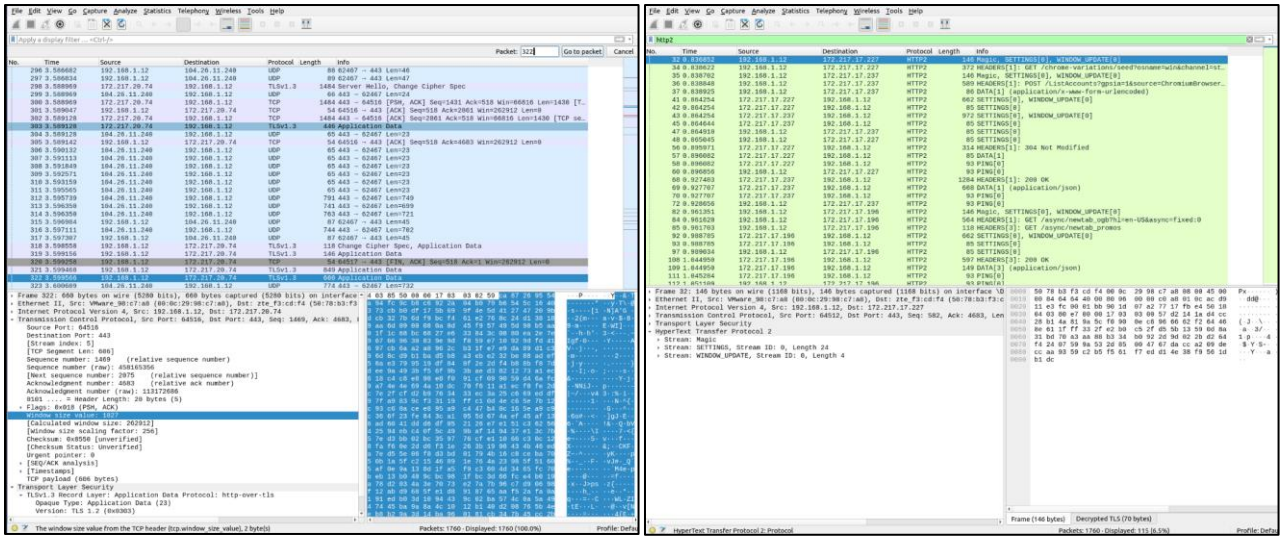
The Wireshark "Follow TCP Stream" feature was used to reassemble the full conversation between the client and the server, making it easy to read the exchanged data as a continuous block of text rather than individual packets.



Decrypting HTTPS Traffic

HTTPS encrypts web traffic so that it cannot be read by anyone intercepting the connection. However, if the encryption key used during a session is available, it is possible to decrypt the captured packets and read the content as if it were plain text.

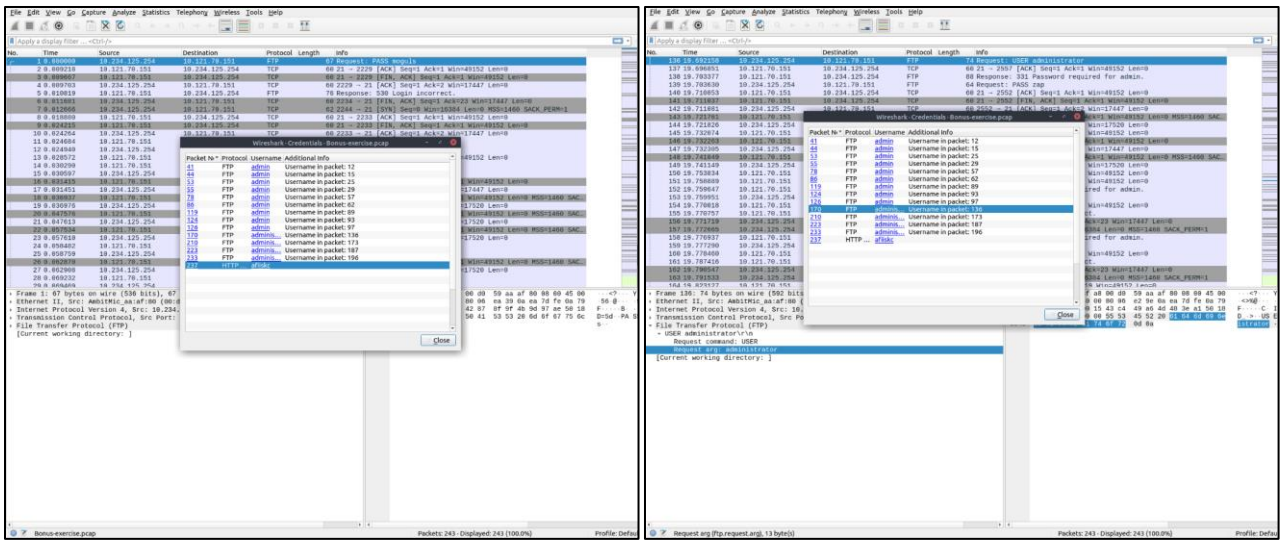
A key log file was provided for this section. It was loaded into Wireshark through the protocol preferences settings, which told Wireshark how to decode the encrypted packets. Once decrypted, the HTTPS traffic became readable, revealing the websites visited, the data exchanged, and any credentials passed over what would otherwise appear to be a secure connection.



Hunting for Exposed Credentials

As a dedicated exercise, Wireshark's built-in credential detection feature was used to scan the entire traffic capture for any usernames and passwords transmitted without encryption. This tool checks for credentials across multiple protocols simultaneously and presents them in a single view, including the protocol used, the source IP, and the credentials themselves.

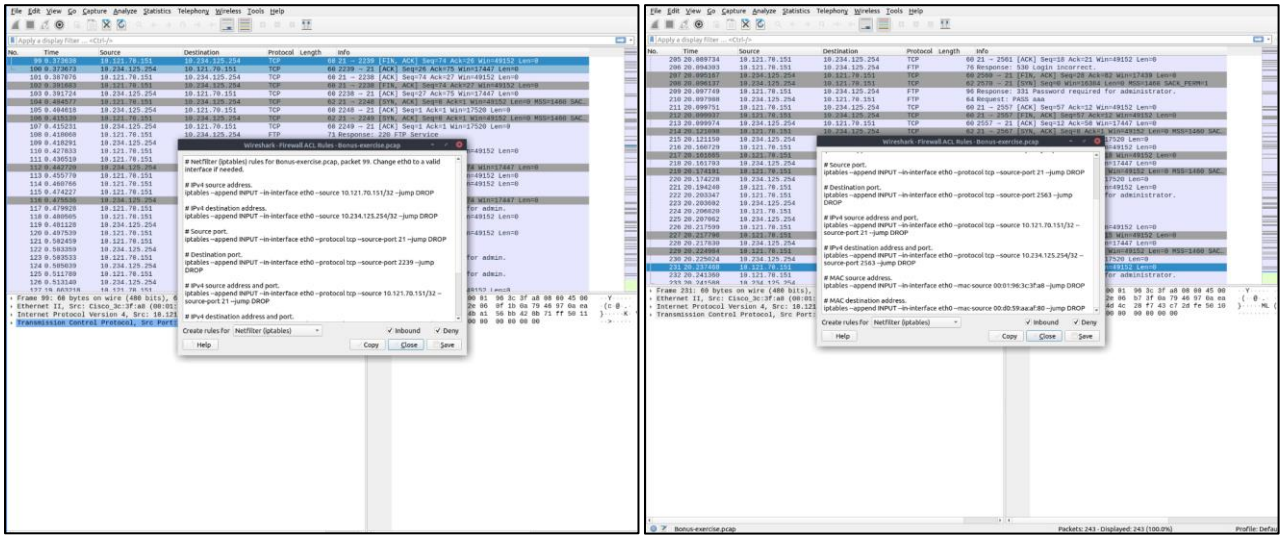
Several sets of credentials were found across different protocols, highlighting how much sensitive information can be exposed when unencrypted communication is used — even briefly.



Generating Firewall Rules

The final activity involved translating investigative findings into practical defensive action. Wireshark includes a built-in tool that can automatically generate firewall blocking rules based on selected packets. This converts the results of an investigation directly into configuration syntax that can be applied to a firewall to block the identified malicious traffic.

Specific suspicious packets were selected, and the firewall rule generator was used to produce rules in IPFirewall (ipfw) syntax. This demonstrated how network forensics findings can move beyond documentation and directly feed into active defense measures.



3. Summary

This lab provided a complete walkthrough of how network traffic can be used as evidence to detect, investigate, and respond to malicious activity. Working entirely from pre-captured traffic files in Wireshark, several different attack types were identified, from early-stage reconnaissance scanning to active interception of communications, to the exposure of login credentials across unencrypted connections.

A key takeaway from this lab is that many attacks leave clear and readable traces in network traffic, but only if you know what to look for. Recognizing the patterns of a port scan, understanding what abnormal DNS traffic looks like, or knowing that FTP sends passwords in plain text are all skills that directly translate to real-world security monitoring.

The lab also demonstrated that investigation does not stop at detection. The ability to take a suspicious packet and immediately generate a firewall rule to block it bridges the gap between finding a threat and acting on it, which is ultimately the goal of any security analyst.

Area Covered	Key Takeaway
Network Scanning	Identified TCP Connect, SYN, and UDP scan patterns used by attackers to map a network before striking.
ARP Poisoning	Detected fake ARP messages are used to intercept traffic between two devices on the same network.
Host Identification	Extracted device names, IP addresses, and user account names from DHCP, NetBIOS, and Kerberos traffic.
Tunneling Detection	Found data hidden inside DNS and ICMP packets — a method used to bypass firewall restrictions.
Cleartext Protocols	Read FTP and HTTP credentials and file transfer data directly from unencrypted packet payloads.
HTTPS Decryption	Decrypted encrypted web traffic using an SSL key log file to expose session content.
Credential Hunting	Used Wireshark's built-in tool to surface all exposed credentials across multiple protocols in one view.
Firewall Rule Generation	Produced ready-to-deploy firewall deny rules from identified malicious traffic using Wireshark's ACL generator.